ESP32-BASED SMART NAVIGATION SYSTEM WITH API-BASED ROUTE GUIDANCE

Aim

The aim of this project is to design and develop an ESP32-based Smart Navigation System that utilizes WiFi connectivity and API-based services to provide route guidance. The system displays navigation instructions and provides audio alerts to assist users in reaching their destination efficiently.

Objectives

- To develop a compact navigation system using ESP32
- To fetch route data using API services
- To display navigation instructions on an OLED screen
- To provide audio alerts using a buzzer
- To integrate IoT and embedded systems for real-time applications
- To design a low-cost and portable navigation solution

Components

- ESP32 Development Board
- OLED Display (SSD1306)
- Buzzer
- Breadboard
- Jumper Wires
- Power Supply (USB/Battery)

Theory

The ESP32-Based Smart Navigation System works based on **IoT communication and API data processing**.
The ESP32 connects to a WiFi network and sends an HTTP request to the navigation API (OpenRouteService). The API responds with route data in JSON format. This data includes step-by-step navigation instructions.
The system processes the JSON data and extracts useful navigation steps. These instructions are displayed on the OLED screen. At the same time, a buzzer provides alerts to notify the user about direction changes.
This approach eliminates the need for complex GPS systems by relying on internet-based route services.

Explanation

- The ESP32 connects to a WiFi network
- It sends a request to the navigation API with source and destination coordinates
- The API returns route data in JSON format
- The ESP32 parses the data and extracts navigation instructions
- Instructions are displayed on the OLED screen
- The buzzer gives an alert for each step

Prototype

The prototype is designed as a **compact and portable navigation device**.

- The ESP32 acts as the main controller of the system
- The OLED display is mounted to show navigation instructions clearly
- The buzzer is connected to provide audio alerts
- A power source (USB or battery) ensures portability
- The system is assembled on a breadboard for easy testing and modification

Working Setup

All components are connected as follows:

- OLED Display → ESP32 (I2C communication)
- ESP32 → Buzzer (for alerts)
- Power Supply → ESP32

OLED Display:

- VCC → 3.3V
- GND → GND
- SDA → GPIO 21
- SCL → GPIO 22

Buzzer:

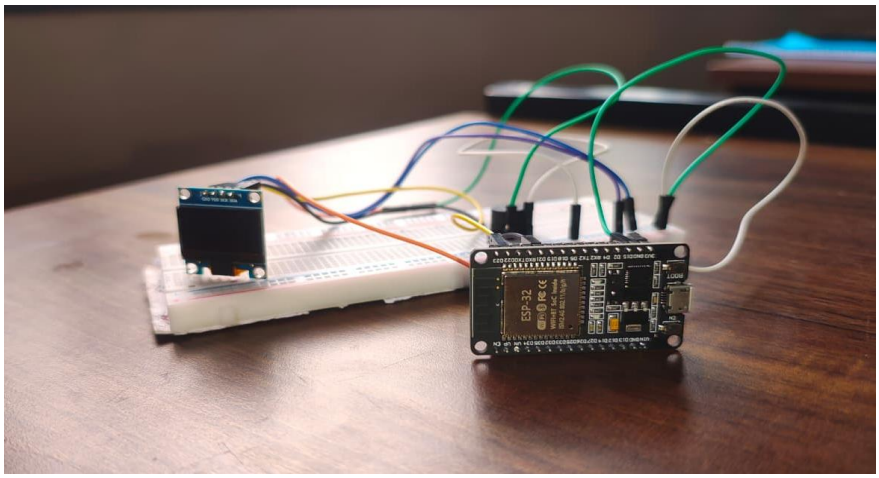
- Positive → GPIO 25
- Negative → GND

When powered ON, the ESP32 connects to WiFi, fetches navigation data, and displays instructions while generating alerts.

Design Features

- Compact and portable design
- Low power consumption
- Real-time navigation updates
- Simple and easy-to-use interface
- Cost-effective implementation
- Expandable for future enhancements

Picture



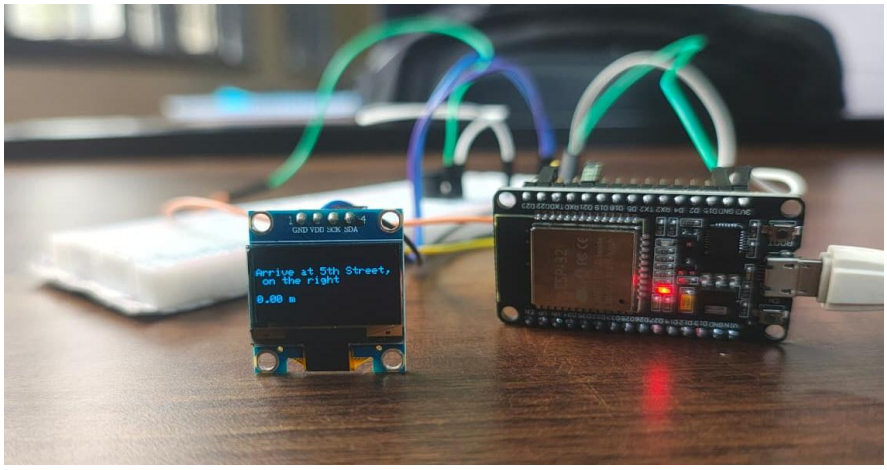
Program (Arduino Code)

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <WiFiClientSecure.h>
#include <ArduinoJson.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

// WiFi
const char* ssid = "Vivo Y10";
const char* password = "hehehe247";

// API Key
String apiKey =
"eyJvcmciOiI1YjNjZTM1OTc4NTExMTAwMDFjZjYyNDgiLCJpZCI6Ijk2ODU3MTUxOGE2MTR
iNWU5OWNkODg2NTk2YTg4M2FjIiwiaCI6Im11cm11cjY0In0=";

// Coordinates
String start = "77.5946,12.9716";
String end   = "80.2707,13.0827";

// OLED
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

// 🔊 BUZZER PIN
#define BUZZER 25

// 🔊 BUZZER FUNCTION
void beep() {
  digitalWrite(BUZZER, HIGH);
  delay(120);
  digitalWrite(BUZZER, LOW);
}

void setup() {

  Serial.begin(115200);

  // 🔊 INIT BUZZER
  pinMode(BUZZER, OUTPUT);
  digitalWrite(BUZZER, LOW);

  // OLED INIT
  if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println("OLED not found");
    while(true);
  }

  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(WHITE);
```

```
display.setCursor(0,0);
display.println("Starting...");
display.display();

// WiFi
WiFi.begin(ssid, password);

display.setCursor(0,20);
display.println("Connecting WiFi");
display.display();

while (WiFi.status() != WL_CONNECTED) {
  delay(500);
  Serial.print(".");
}

display.clearDisplay();
display.setCursor(0,0);
display.println("WiFi Connected");
display.display();
delay(1000);

WiFiClientSecure client;
client.setInsecure();

HTTPClient http;

String url =
  "https://api.openrouteservice.org/v2/directions/driving-car?api_key=" +
  apiKey +
  "&start=" + start +
  "&end=" + end;

http.begin(client, url);
int httpCode = http.GET();
if (httpCode > 0) {
  String payload = http.getString();
  DynamicJsonDocument doc(60000);
  deserializeJson(doc, payload);

  JsonArray steps =
    doc["features"][0]["properties"]["segments"][0]["steps"];

  for (JsonObject step : steps) {

    const char* instruction = step["instruction"];
    float distance = step["distance"];

    // SERIAL OUTPUT
    Serial.println(instruction);

    // OLED OUTPUT
    display.clearDisplay();

    display.setCursor(0,0);
```

```
      display.println("NAVIGATION");

      display.setCursor(0,15);
      display.println(instruction);

      display.setCursor(0,40);
      display.print(distance);
      display.println(" m");

      display.display();

      //  BUZZER SOUND ON EVERY STEP CHANGE
      beep();

      delay(3000);
    }

    //  FINAL DESTINATION SOUND
    for(int i = 0; i < 5; i++) {
      beep();
      delay(150);
    }

  } else {

    display.clearDisplay();
    display.setCursor(0,0);
    display.println("API FAILED");
    display.display();

    Serial.println("API request failed");
  }

  http.end();
}

void loop() {}
```

Output



Result

The ESP32-Based Smart Navigation System was successfully implemented and tested. The system was able to connect to WiFi, retrieve route data from the API, and display navigation instructions on the OLED screen. Audio alerts were generated using the buzzer for each navigation step.
The project demonstrates effective integration of IoT, embedded systems, and API-based communication, providing a compact and cost-effective navigation solution.